

Міністерство освіти і науки України  
Національний технічний університет  
України

«Київський політехнічний інститут  
імені Ігоря Сікорського»

Факультет інформатики та  
обчислювальної техніки

Кафедра обчислювальної техніки

**Лабораторна робота №4**

**з дисципліни «Об'єктно-  
орієнтоване програмування»**

**на тему**

# «Вдосконалення структури коду графічного редактора об'єктів»

## Виконав:

Степаненко Денис  
студент групи ІМ-051  
номер у списку групи: 13

## Перевірів:

Рекечинський Дмитро Олександрович

Київ 2026

---

### Завдання

1. Створити у середовищі розробки проєкт з ім'ям Lab4.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. Налогодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

---

### Завдання згідно варіанту

Номер у списку групи: **13 (непарний)**

---

| Вимога |
|--------|
|--------|

---

| Реалізація |
|------------|
|------------|

|                               |
|-------------------------------|
| Об'єкт редактора (непарний №) |
|-------------------------------|

**глобальний статичний** `static MyEditor g_editor;`

Кольори/стилі фігур

як у Lab 3 (`rect` — без заповнення, два кути; `ellipse` — світло-зелене заповнення, центр→кут)

«Гумовий» слід

**пунктирна** лінія для усіх фігур (`PS_DOT`)

Фігури з Lab2/3

точка, лінія, прямокутник, еліпс

Нові фігури

**лінія з кружечками** (`Line00Shape`), **каркас куба** (`CubeShape`)

Спосіб реалізації нових

**множинне успадкування:** `LineShape+EllipseShape`, `LineShape+RectShape`

Делегування `Show()`

лінії → `LineShape::Show`, кружечки → `EllipseShape::Show`, грані куба → `RectShape::Show`, ребра → `LineShape::Show`

Toolbar

6 кнопок-іконок з підказками, по одній на тип фігури

Звіт

діаграма класів, фрагменти коду, скріншоти, висновки

---

## Структура програми

Програма продовжує графічний редактор Lab3. Бонусні інструменти Lab3 (трикутник, вибір, губка, діалог кольору) прибрано — умова Lab4 задає рівно шість типів фігур і кнопку Toolbar для кожного типу.

---

### Файл

---

#### Призначення

Lab4.cpp

`WinMain`, `WndProc` — делегує повідомлення у глобальний статичний `g_editor`  
`editor.h/.cpp`

клас `MyEditor`: стан редактора + обробники повідомлень

`toolbar.h/.cpp`

Toolbar: 6 іконок-кнопок, підказки

`shape.h/.cpp`

абстрактний `Shape` + спільні поля координат/кольорів/`m_penStyle`

`point.*`, `line.*`, `rect.*`, `ellipse.*`

базові фігури (`Line/Rect/Ellipse` — `virtual public Shape`)

`lineoo.*`

`Line00Shape` : `LineShape`, `EllipseShape` — лінія з кружечками

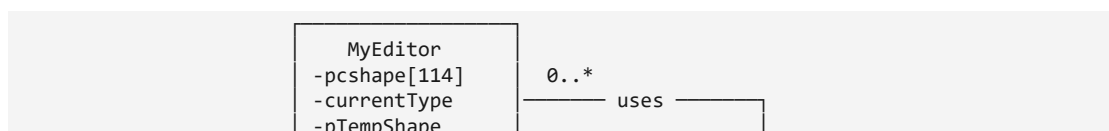
`cube.*`

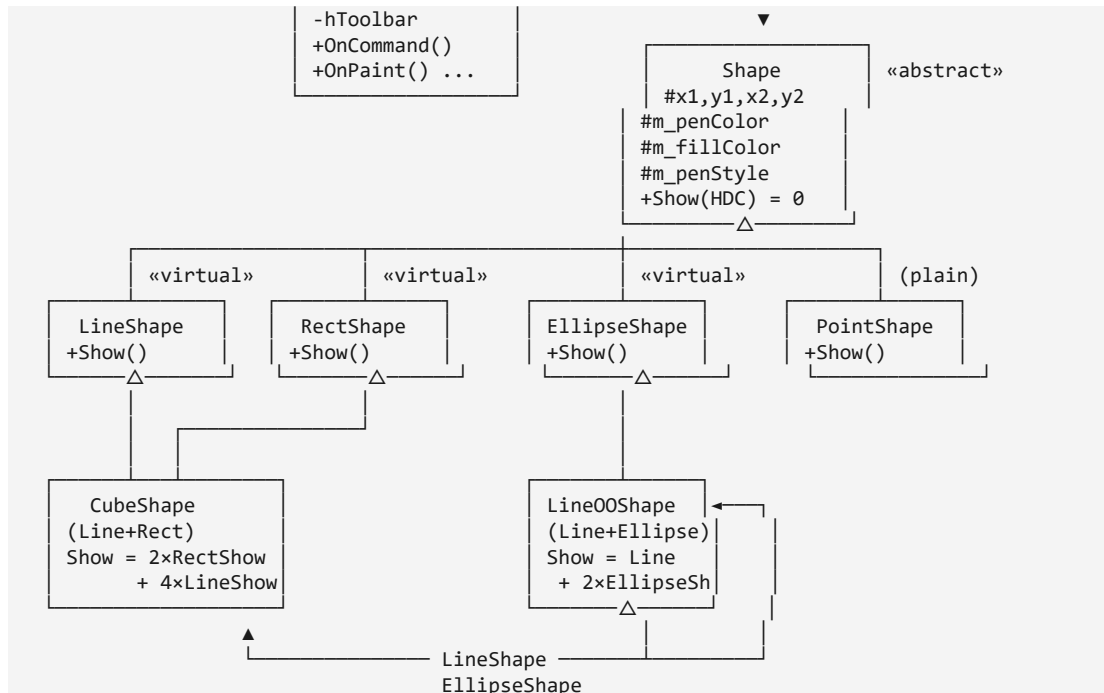
`CubeShape` : `LineShape`, `RectShape` — каркас куба

`resource.h`, `Lab4.rc`

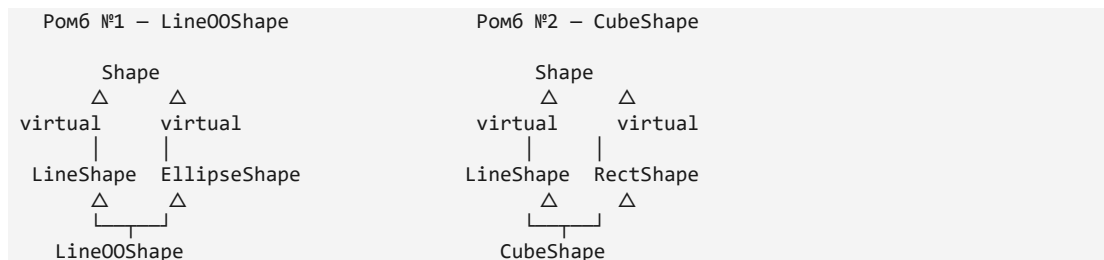
ідентифікатори команд, меню

## Діаграма класів





Та сама ієрархія у вигляді двох ромбів:



Без virtual у кожному ромбі було би **два** примірники Shape (координати x1..y2 дублюються, Show() неоднозначний, приведення CubeShape\* → Shape\* не компілюється). virtual public Shape у LineShape/RectShape/EllipseShape лишає один спільний базовий субоб'єкт; PointShape лишається звичайним успадкуванням — він не є батьком жодного ромба.

## Ключові фрагменти коду

### Глобальний статичний об'єкт MyEditor (Lab4.cpp)

```
static MyEditor g_editor; // непарний варіант: статичний, не new/delete
```

Об'єкт створюється ще до winMain (при старті програми), деструктор спрацьовує при завершенні — ручне керування пам'яттю не потрібне.

### Множинне успадкування — лінія з кружечками (lineoo.h)

```
class Line00Shape : public LineShape, public EllipseShape
{
public:
    Line00Shape(int x1 = 0, int y1 = 0, int x2 = 0, int y2 = 0)
        : Shape(x1, y1, x2, y2), // most-derived ініціалізує virtual base
          LineShape(x1, y1, x2, y2),
```

## Пунктирний «гумовий» слід (shape.h + editor.cpp)

Стиль пера винесено в поле базового класу — усі Show створюють CreatePen(m\_penStyle, ...):

```
// Shape
int m_penStyle; // PS_SOLID за замовчуванням
void SetPenStyle(int style) { m_penStyle = style; }
// MyEditor::OnLButtonDown — preview-об'єкт
pTempShape->SetPenColor(RGB(0, 0, 255));
pTempShape->SetPenStyle(PS_DOT); // пунктирний слід
pTempShape->ClearFill();
```

Поле живе у спільному Shape-субоб'єкті, тому при preview навіть складені МІ-фігури (лінія з кружечками, куб) цілком малюються пунктиром.

## Шість кнопок Toolbar (toolbar.cpp)

```
#define BTN_COUNT 6
const int ids[BTN_COUNT] = { IDM_POINT, IDM_LINE, IDM_RECT,
                             IDM_ELLIPSE, IDM_LINEOO, IDM_CUBE };
```

Кожна кнопка — згенерована GDI-іконка 16×16 (для нових фігур намальовано міні-гліфи: лінія з кружечками та каркас куба), TBSTYLE\_CHECKGROUP, підказки через TTN\_NEEDTEXT.

---

## Результати роботи

### Головне вікно з Toolbar

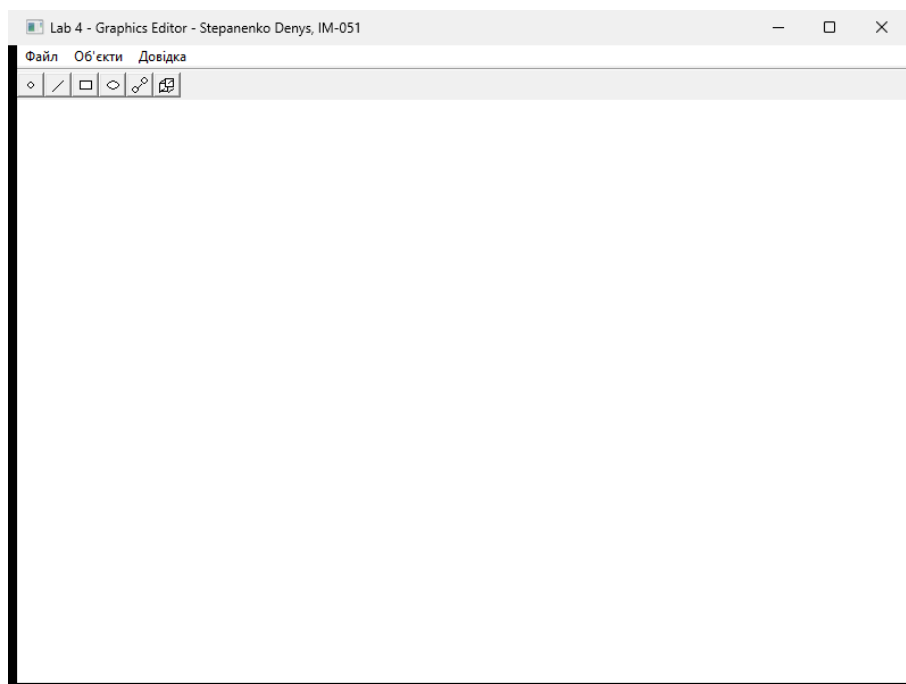


Рис. 1. Головне вікно: 6 кнопок-іконок (точка, лінія, прямокутник, еліпс, лінія з кружечками, каркас куба), меню «Об'єкти» між «Файл» та «Довідка»

## Усі шість типів фігур

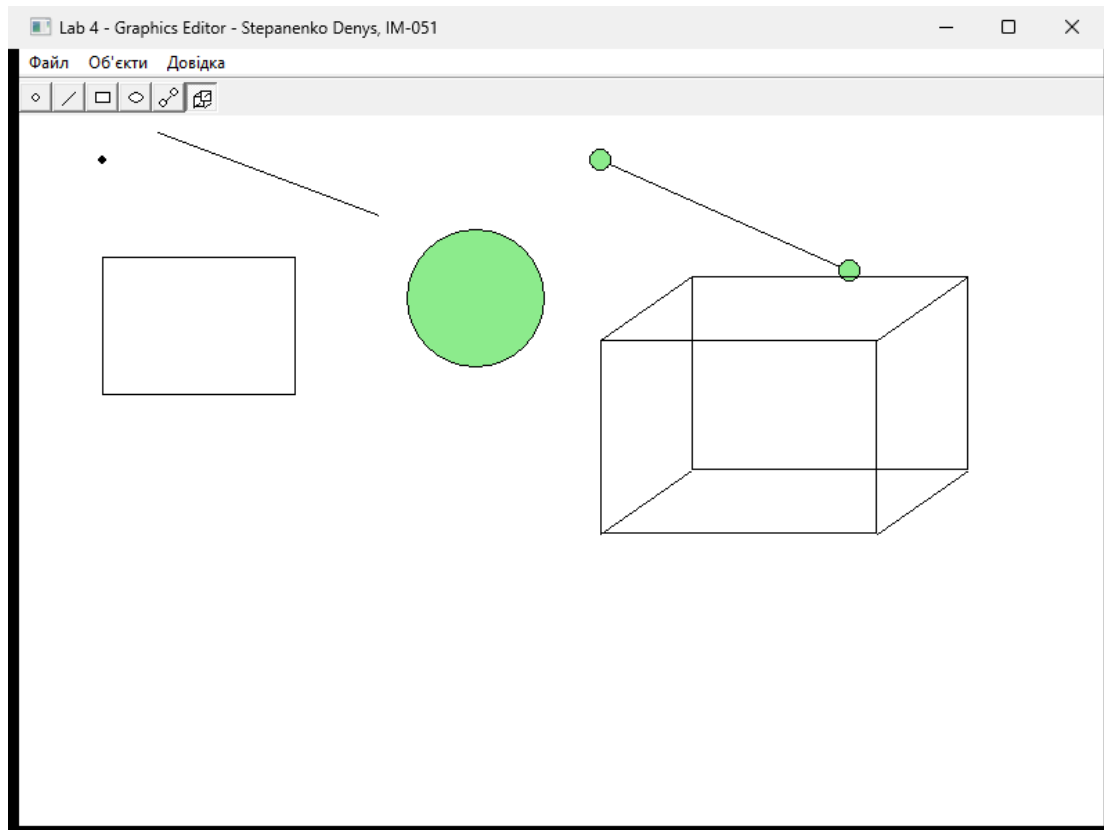


Рис. 2. Об'єкти всіх типів: точка, лінія, прямокутник (без заливки), еліпс (світло-зелене заповнення), лінія з кружечками на кінцях, каркас куба (дві грані + ребра)

### Пунктирний «гумовий» слід

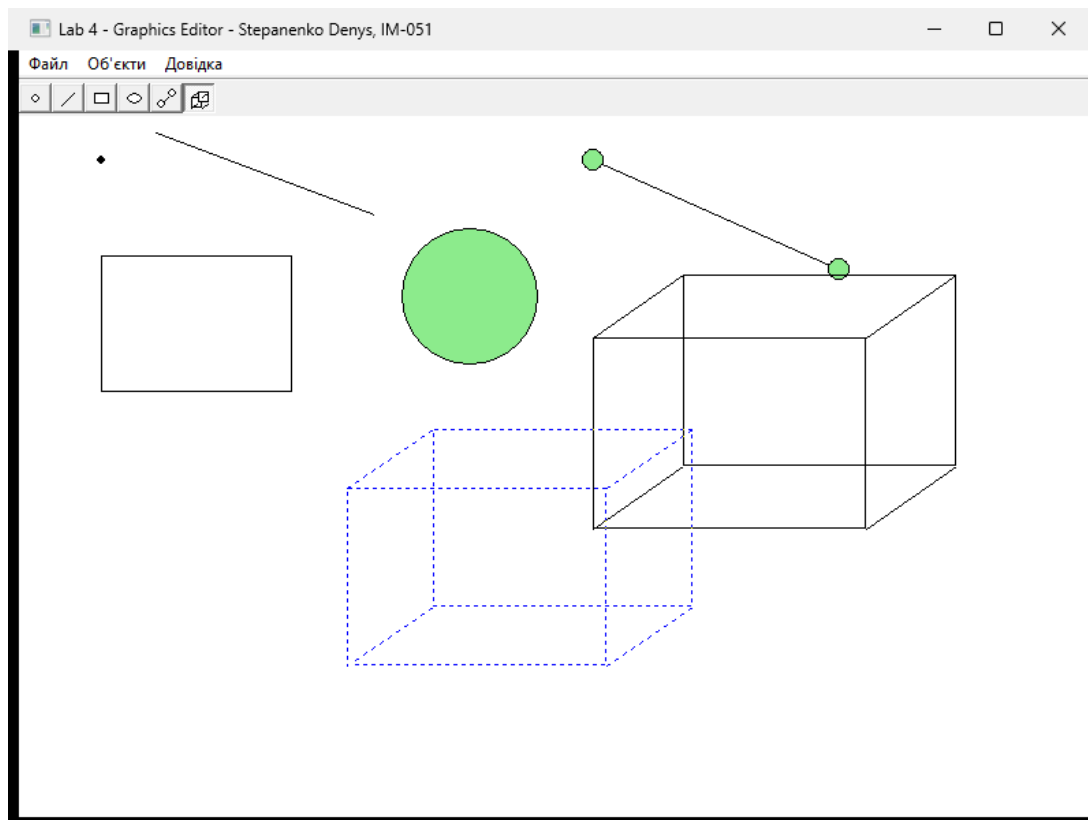


Рис. 3. Під час вводу каркасу куба preview малюється пунктирною синьою лінією (ps\_dot) — усі частини складеної фігури пунктирні

## Висновки

У лабораторній роботі виконано модернізацію структури коду графічного редактора згідно варіанту (непарний №13): об'єкт редактора — глобальний статичний `MyEditor g_editor`, «гумовий» слід усіх фігур — пунктирний (`ps_dot`), додано два нові типи фігур, реалізовані саме **множинним успадкуванням**: `Line00Shape : LineShape, EllipseShape` та `CubeShape : LineShape, RectShape`.

Ключова проблема — ромбовидне успадкування: у MI-класів `Shape` усічається двічі (через обох батьків), що дублює поля координат і робить `Show()` неоднозначним. Розв'язано віртуальним успадкуванням `virtual public Shape` у проміжних класах — у MI-об'єктів існує один спільний `Shape`-субоб'єкт, а `most-derived` клас ініціалізує його у власному конструкторі. Делегування `LineShape::Show / EllipseShape::Show / RectShape::Show` виконується з тимчасовою підстановкою спільних полів координат — саме той прийом, який вимагає методика.

Додано шосту кнопку `Toolbar` (міні-іконки для нових фігур малюються GDI у бітмапи 16×16), оновлено меню «Об'єкти». Лабораторна дала навички множинного успадкування, розв'язання `diamond`-проблеми через `virtual`, та проектування ієрархії, де нові фігури-комбінації складаються з існуючих `Show` без дублювання коду.

---

## Контрольні запитання

**1. Що таке поліморфізм і як він використаний у цій роботі?** Поліморфізм — здатність об'єктів різних класів відповідати на одне й те саме повідомлення по-своєму. Масив `Shape* pcshape[]` зберігає вказівники на об'єкти різних типів; виклик `pcshape[i]->Show(hdc)` поліморфно диспетчеризується у конкретний `Show` через `vtable`. Те саме з `OnMouseMove` при `preview`.

**2. Обробку яких повідомлень потрібно виконувати для вводу об'єктів?** `WM_LBUTTONDOWN` (початок фігури + `SetCapture`), `WM_MOUSEMOVE` (оновлення другого кута + `InvalidateRect` для `preview`), `WM_LBUTTONUP` (фіксація фігури + `ReleaseCapture`), `WM_PAINT` (перемалювання всіх об'єктів + гумового сліду), `WM_COMMAND` (вибір типу з меню/тулбара), `WM_INITMENUPOPUP` (маркер типу в меню), `WM_NOTIFY/TTN_NEEDTEXT` (підказки кнопок).

**3. Що таке абстрактний клас і скільки їх у цій програмі?** Абстрактний клас — клас із хоча б одним чисто віртуальним методом; об'єкти такого класу створити не можна. У програмі один абстрактний клас — `Shape` (`virtual void Show(HDC) = 0`).

**4. Що таке множинне успадкування і як воно впливає на модульність?** Множинне успадкування — клас успадковує два і більше батьків одночасно (`class Line00Shape : public LineShape, public EllipseShape`). Модульність зростає: нові фігури-комбінації збираються з готових `Show` батьків без копіювання коду — додавання нового типу не вимагає переписування існуючих модулів.

**5. Що таке ромбічне успадкування і які проблеми воно спричиняє?** Ромб — коли клас має спільного предка двома шляхами: `Line00Shape → LineShape → Shape` і `Line00Shape → EllipseShape → Shape`. Без `virtual` об'єкт містить **два** `Shape`-субоб'єкти: поля `x1.y2` дублюються, доступ до них неоднозначний (`ambiguous`), приведення до `Shape*` не компілюється. Розв'язок — `virtual public Shape` у проміжних класах: спільний базовий субоб'єкт один, а ініціалізує його конструктор самого похідного класу.

**6. Як додати нові кнопки у Toolbar?** Згенерувати іконку (тут — намалювати GDI у бітмапі 16×16), передати її в тулбар повідомленням `TB_ADDBITMAP` (отримуємо індекс), додати `TB_BUTTON` з `iBitmap=idx`, `idCommand=IDM_новоїКоманди`, `fsStyle=TBSTYLE_CHECKGROUP` повідомленням `TB_ADDBUTTONS`, додати текст підказки у `ToolTipText()` і пункт у меню «Об'єкти» з тим самим ID — тоді кнопка й пункт меню працюють через спільний `WM_COMMAND`.